

COL216 ASSIGNMENT 3

1. Ayush Pandey 2023CS10597

2. Chowan Srichari 2022CS

Overview

This report documents a multi-core cache simulator implemented in C++ that models the MESI cache coherence protocol. The simulator processes memory access traces for four cores, simulating cache operations including reads, writes, evictions, and writebacks, while tracking performance statistics such as miss rates, bus traffic, and invalidations.

System Components

The simulator consists of the following key classes:

1. **Cache:** Represents a single cache with configurable set index bits, associativity, and block size. It manages cache lines, implements the MESI protocol, and handles bus transactions.
2. **Core:** Simulates a processor core, reading memory access traces and issuing read/write operations to its cache.
3. **BusController:** Manages bus transactions (BUSRD, BUSRDX, FLUSH, INVALIDATE) between caches and memory.
4. **CacheSimulator:** Orchestrates the simulation, coordinating cores and the bus controller, and outputs results.

Flowchart of Code Execution

Below is a high-level flowchart of the simulator's execution flow:

graph TD

Code Description

The provided C++ code implements a cache simulator with the following features:

- **MESI Protocol:** Supports INVALID, SHARED, EXCLUSIVE, and MODIFIED states for cache coherence.
- **Cache Configuration:** Configurable via command-line arguments for set index bits (-s), associativity (-E), and block size (-b).
- **Trace Processing:** Reads memory access traces (e.g., app1_proc0.trace) for each core, with operations specified as R (read) or W (write) followed by a hexadecimal address.
- **Statistics Tracking:** Tracks per-core metrics (reads, writes, cycles, misses, evictions, writebacks) and global metrics (invalidations, bus traffic).
- **Bus Transactions:** Handles cache-to-cache and cache-to-memory data transfers, with realistic cycle counts (e.g., 2 cycles per word for cache-to-cache, 100 cycles for memory access).

Key Functions

- **Cache::parseAddress:** Extracts tag, set index, and block offset from a memory address.
- **Cache::allocateLine:** Allocates a cache line, handling evictions and writebacks using LRU replacement.
- **Core::processRead/processWrite:** Processes read/write operations, issuing bus requests on cache misses.
- **BusController::processBusCycle:** Manages bus transactions, coordinating data transfers and cache state updates.
- **CacheSimulator::run:** Runs the simulation loop until all cores complete their traces.

Simulation Flow

1. Initialization:

- Parse command-line arguments to configure cache parameters and trace file prefix.
- Create four cores, each with a cache and a trace file.

2. Simulation Loop:

- Increment the cycle count.
- For each core:
 - If not waiting or done, process the current operation (read/write).
 - Update idle cycles if waiting.
- Process bus transactions via the bus controller.

3. Completion:

- Continue until all cores have processed their traces and are not waiting.
- Output results to a file or stdout, including cache configuration, per-core statistics, and global statistics.

Usage

The simulator is executed with the following command:

```
./simulator -t <trace_prefix> -s <set_index_bits> -E <associativity> -b  
<block_bits> [-o <output_file>]
```

Example:

```
./simulator -t app1 -s 6 -E 2 -b 5 -o results.txt
```

Output

The output includes:

- Cache configuration (size, associativity, block size).

- Per-core statistics:
 - Number of instructions (reads and writes).
 - Total and idle cycles.
 - Cache miss rate.
 - Evictions and writebacks.
- Global statistics:
 - Number of invalidations.
 - Total bus traffic (in bytes).

Conclusion

The cache simulator effectively models a multi-core system with MESI cache coherence, providing detailed performance metrics. The implementation is modular, with clear separation of concerns between cache management, core operations, bus control, and simulation orchestration.